

Examen de programmation

Durée: 2h

Version: **A**

Instructions

- Répondez sur la feuille de réponse séparée **à votre nom**. **Aucune** réponse écrite sur le présent énoncé ne sera prise en compte dans la correction
- Répondez en noircissant la case de la réponse avec un **stylo ou un feutre** (pas de crayon).
- Noircissez aussi la version (**A** ou **B**) de votre énoncé sur la feuille de réponse.
- Pour corriger une réponse, **effacez complètement** la mauvaise case avec un typex avant d'en noircir une autre.
- Vous ne devez pas remettre l'énoncé. Vous pouvez donc l'utiliser comme brouillon.
- Il n'y a pas de points négatifs.
- **Aucun** document n'est autorisé.

Question 1 (1 point)

Laquelle des propositions suivantes est fausse ?

- A) La mémoire RAM est volatile.
- B) Le signe = sert à assigner une valeur à une variable.
- C) Le mot-clé return permet de définir la valeur de retour d'une fonction.
- D) L'opérateur % calcule le reste de la division entière.
- E) Les instructions d'un programme sont toutes exécutées en même temps.
- F) La valeur renvoyée par l'opérateur != est un booléen.
- G) Le while exécute son bloc tant que sa condition est vraie.
- H) Une expression est un morceau de code qui produit une valeur.
- I) L'interpréteur Python est indispensable pour exécuter un programmes Python.
- J) Les indentations peuvent se faire avec des tabulations ou des espaces.

Question 2 (1 point)

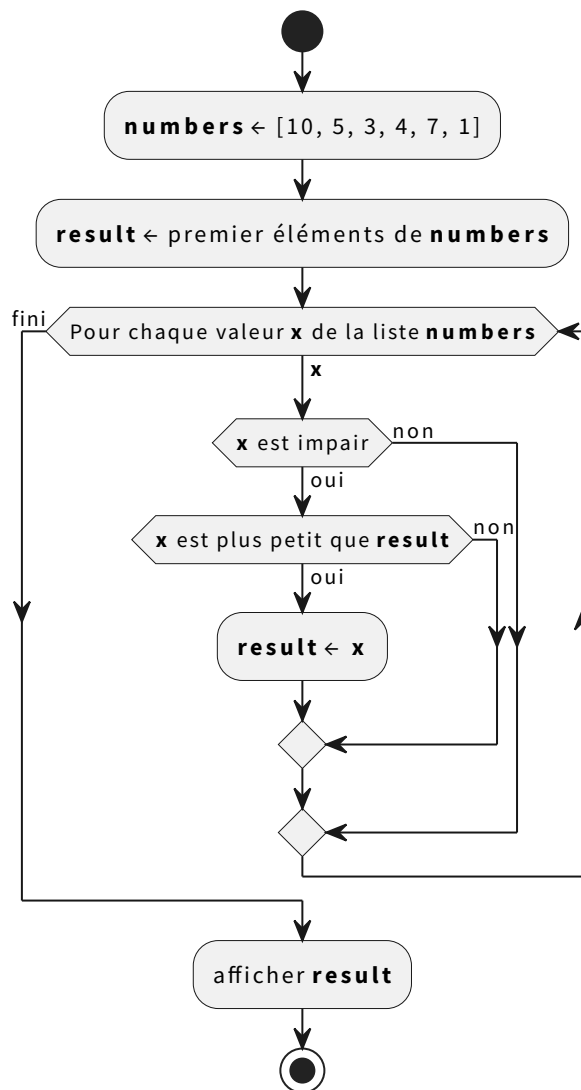
Que va afficher le code suivant ?

```
1 l = [1, 2, 3]
2 print(l[-1])
```

- A) [1, 2, 3]
- B) 123
- C) 1
- D) 2
- E) 3
- F) -1
- G) "l[-1]"
- H) None
- I) Message d'erreur

Question 3 (2 points)

Que va afficher le programme représenté par le diagramme suivant ?



A) 10

B) 5

C) 3

D) 4

E) 7

F) 1

G) None

H) "result"

I) "x"

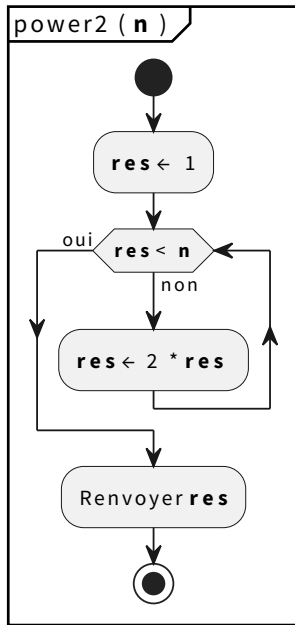
J) [10, 5, 3, 4, 7, 1]

Question 4 (3 points)

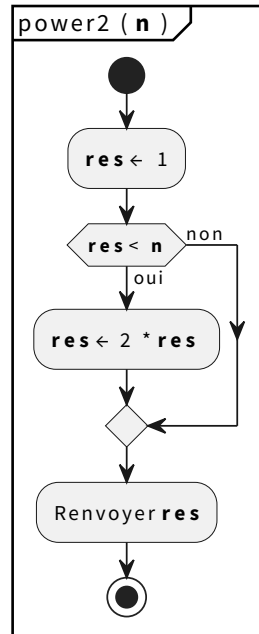
On souhaite une fonction nommée `power2()` qui reçoit un nombre `n` positif en paramètres et qui renvoie la première puissance de 2 supérieure ou égale à `n`.

Quel diagramme décrit correctement la fonction `power2()` ?

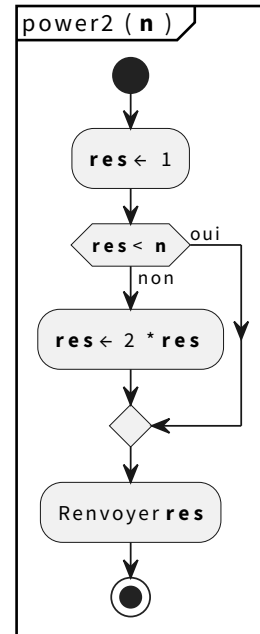
A)



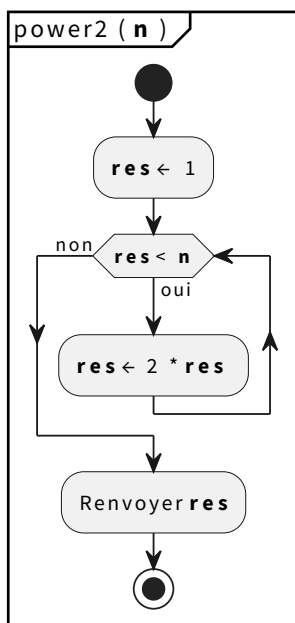
B)



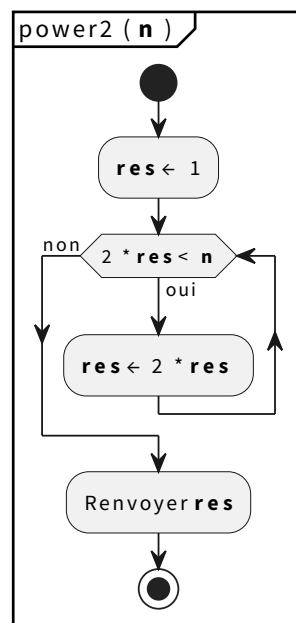
C)



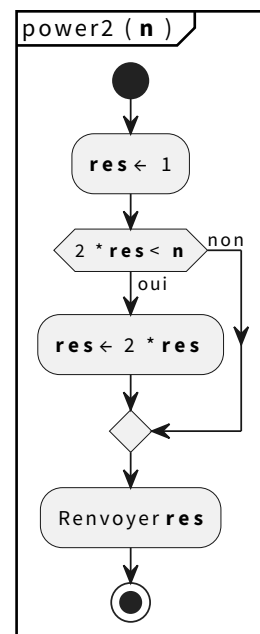
D)



E)



F)



Question 5 (2 points)

Que va afficher le code suivant ?

```
1 def remove_vowel(word):
2     res = ""
3     for c in word:
4         if c not in "aeiouAEIOU":
5             res += c
6     return res
7
8 def big(word):
9     return len(word) > 4
10
11 L = ["pierre", "paul", "jacques"]
12
13 print(list(map(remove_vowel, filter(big, L))))
```

A) <map object at 0x104c5b940>

B) ['prrr', 'jcqs']

C) ['prrr', 'pl', 'jcqs']

D) []

E) <filter object at 0x104c5b940>

F) ['iee', 'au', 'aue']

G) ['iee', 'aue']

H) ['', '']

I) ['', '', '']

J) Aucune des propositions

Question 6 (3 points)

Dans le code suivant, la fonction `sums(table)` reçoit une liste de liste de nombres en paramètre (exemple: `[[1, 2, 3], [4, 5, 6]]`). La fonction doit renvoyer la liste des sommes des sous-listes (c-à-d `[5, 15]` pour l'exemple).

Malheureusement, le code plante lorsqu'on l'exécute.

```
1  def sums(table):
2      res = []
3      for row in table:
4          for item in row:
5              sum += item
6              res.append(sum)
7      return res
8
9  table = [[1, 2, 3], [4, 5, 6]]
10
11 print(sums(table)) # devrait afficher [6, 15]
```

Le message d'erreur est le suivant :

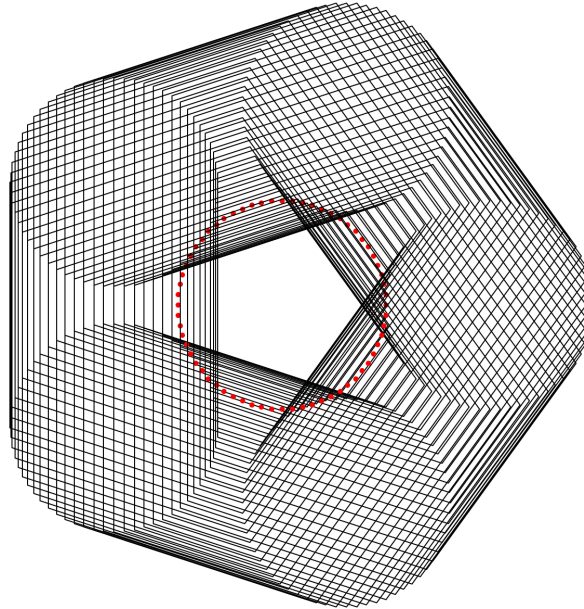
```
Traceback (most recent call last):
  File "/Users/lur/Desktop/exo.py", line 11, in <module>
    print(sums(table))
    ~~~~~^~~~~~
  File "/Users/lur/Desktop/exo.py", line 5, in sums
    sum += item
    ^^
UnboundLocalError: cannot access local variable 'sum' where it is not associated with a value
```

Que faut-il changer pour corriger la fonction ? (les → indiquent le niveau d'indentation)

- A) ligne 2: → `res = 0`
- B) entre les lignes 2 et 3: → `sum = 0`
- C) ligne 3: → `for res in table:`
- D) entre les lignes 3 et 4: → → `sum = 0`
- E) ligne 4: → → `for item in res:`
- F) entre les lignes 4 et 5: → → `item = res[-1]`
- G) ligne 5: → `sum += item[0]`
- H) ligne 6: → `res.append(item)`
- I) ligne 7: → `return res[-1]`
- J) Aucune proposition n'est correcte

Question 7 (4 points)

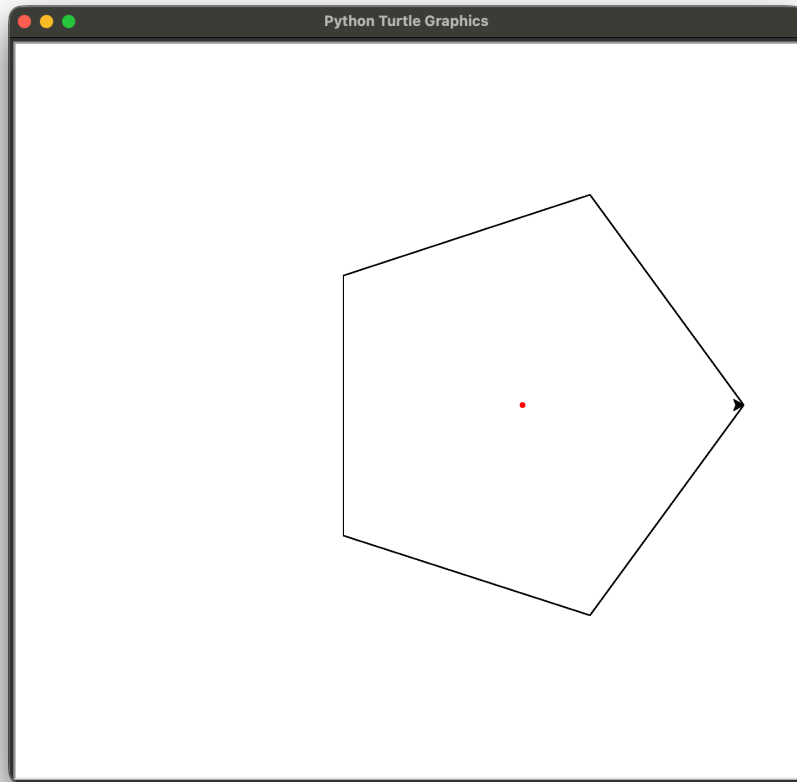
Le dessin turtle suivant est composé de 64 pentagones réguliers inscrits dans des cercles de rayons 200 pixels. Les centres de ces cercles sont affichés en rouge et disposés sur un cercle de rayon 100 pixels.



Voici le programme qui aurait dû l'afficher:

```
1  from turtle import up, goto, down, done, dot, color
2  from math import cos, sin, pi
3
4  def polygon(center, nb_sides, radius):
5      x, y = center
6      angle = 2 * pi / nb_sides
7      up()
8      goto(x, y)
9      color("red")
10     dot() # Affiche un point
11     color("black")
12     goto(x + radius, y)
13     down()
14     for i in range(1, nb_sides + 1):
15         goto(x + radius * cos(i * angle), y + radius * sin(i * angle))
16
17 def spiro(center, radius, count, nb_sides, polygon_radius):
18     x, y = center
19     angle = 2 * pi / count
20     for i in range(count):
21         polygon(
22             (x + radius * cos(angle), y + radius * sin(angle)),
23             nb_sides,
24             polygon_radius,
25         )
26
27 spiro((0, 0), 100, 64, 5, 200)
28 done()
```

Cependant une erreur s'est glissée dans le programme et on obtient ceci :



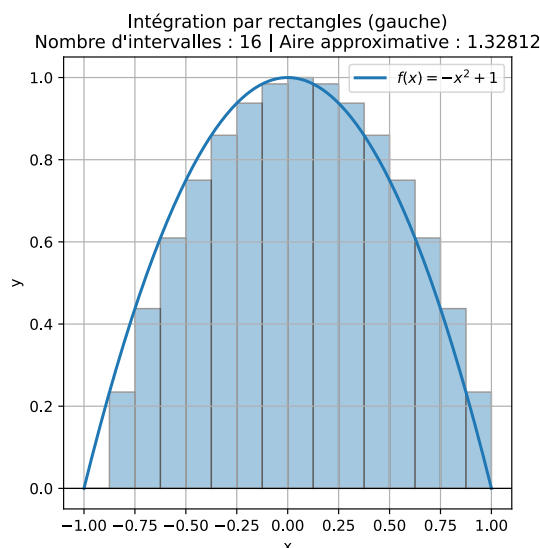
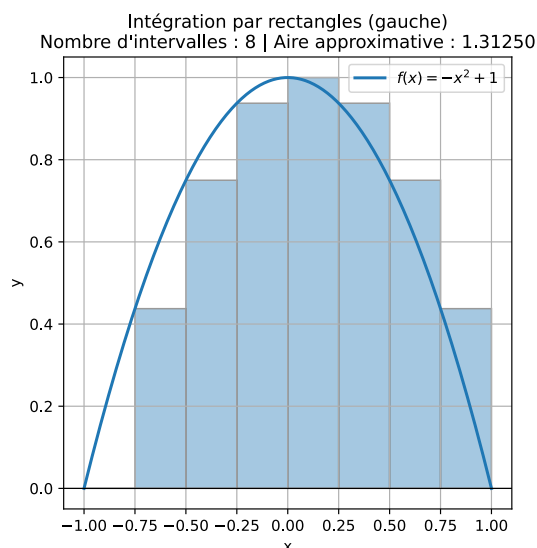
Que faut-il changer pour corriger la fonction ? (*les → indiquent le niveau d'indentation*)

- A) Ligne 6: → `angle = 2 * pi * radius / nb_sides`
- B) Ligne 12: → `goto(x + radius, y + radius)`
- C) Ligne 14: → `for i in range(nb_sides):`
- D) Ligne 15: → → `goto(x + radius * cos(angle), y + radius * sin(angle))`
- E) Ligne 19: → `angle = 2 * pi * radius / count`
- F) Ligne 20: → `for i in range(1, count + 1):`
- G) Ligne 22: → → → `(x + radius * cos(i * angle), y + radius * sin(i * angle)),`
- H) Ligne 27: `spiro(0, 0, 100, 64, 5, 200)`
- I) Aucune proposition n'est correcte

Intégrales définies

Une intégrale définie est la mesure de l'aire située sous la courbe d'une fonction entre deux bornes. Pour obtenir une approximation de cette aire, on peut la découper en rectangles simples. L'idée est de diviser l'intervalle entre les deux bornes en plusieurs parties, puis de définir un rectangle pour chacune d'elles, dont la hauteur dépend de la valeur de la fonction à gauche du rectangle.

Plus le nombre de rectangles utilisés est grand, plus l'approximation est précise.



Dans le programme de la page suivante, la fonction `quad_n()` calcule l'approximation en utilisant n rectangles. La fonction `quad()` commence avec $n=1$ et raffine l'approximation en doublant n à chaque itération. On arrête de subdiviser quand la différence entre la nouvelle estimation et la précédente devient plus petite que la précision `eps` voulue.

Voici le code du programme où il manque deux instructions :

```
1  def f(x):
2      return -x**2 + 1
3
4  def quad_n(fn, inf, sup, n):
5      # dx est la base d'un rectangle
6      dx = # Expression manquante
7      sum = 0
8      for i in range(n):
9          sum += dx * fn(inf + i * dx)
10     return sum
11
12  def quad(fn, inf, sup, eps=1e-8):
13     n = 1
14     sum = quad_n(fn, inf, sup, n)
15     while True:
16         n *= 2
17         prev = sum
18         sum = quad_n(fn, inf, sup, n)
```

```
19     # Ici l'indentation est la même
20     # qu'à la ligne 18
21
22     if # Condition manquante
23         return sum
24
25     print(quad(f, -1, 1)) # 1,333...
```

Question 8 (2 points)

Dans le programme sur les intégrales définies ci-dessus, quelle proposition choisissez-vous pour l'expression manquante de la ligne 6 ?

- | | |
|------------------------------|----------------------------|
| A) $(fn(sup) - fn(inf)) / n$ | B) $n * sup - n * inf$ |
| C) $fn(n) * (sup - inf)$ | D) $(sup - inf) / fn(n)$ |
| E) $fn(inf - sup) * n$ | F) $(sup - inf) / n$ |
| G) $fn((sup - inf) / n)$ | H) [] |
| I) 0 | J) Aucune des propositions |

Question 9 (2 points)

Dans le programme sur les intégrales définies ci-dessus, quelle proposition choisissez-vous pour la condition manquante de la ligne 22 ?

- | | |
|---------------------------|----------------------------|
| A) $sum > prev$ | B) $abs(sum - prev) < eps$ |
| C) $fn(sum - prev) < eps$ | D) $sum - prev < fn(eps)$ |
| E) $fn(sum) > fn(prev)$ | F) $sum - prev < eps$ |
| G) $sum * eps < prev$ | H) $sum + prev > eps$ |
| I) $fn(sum, prev, n) > 0$ | J) Aucune des propositions |

Brouillon